



Module 5-1

Digital IC Functional Design Overview (Part 3)

cādence®

This page does not contain notes.

Module Objectives

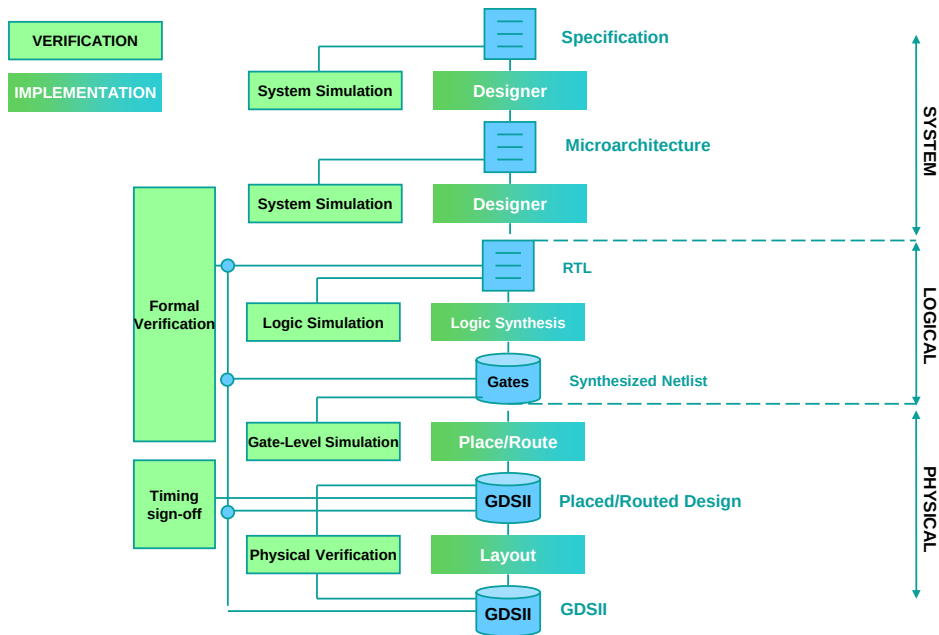
In this module, you will:

- Identify the basic design flow of spec input, coding, and output to synthesis.
- Explain the HDL history.
- Recognize why we were still using RTL and identify the levels of abstraction.



This page does not contain notes.

Complete Basic IC Design Flow



3

© Cadence Design Systems, Inc. All rights reserved.

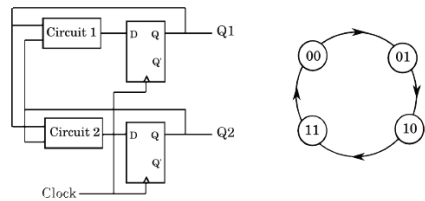


Discuss briefly the three phases of design, including system, logical, and physical.

Example: Design Spec of 2-Bit Synchronous Binary Counter

- The binary counter is an example of a simple synchronous digital circuit. It has no data inputs and no combinational output logic circuit.
- At each clock pulse, the counter takes up a new state and thus goes through a specific count sequence.
- We shall now write a design spec for a binary up-counter with two outputs that go through the following sequence:
 - 00 -> 01 -> 10 -> 11 -> 00 -> etc.
- The counter should have the following inputs:
 - *rst*, which is synchronous, active low
 - *clk*
- The counter should have the following output:
 - count (1:0)

The block diagram, structure, and state transition diagram of a 2-bit binary counter (built with two D-type flip-flops) is of the following form:



The truth table for the circuit is as follows:

Inputs t_n		Outputs t_{n+1}	
Q1	Q2	D1	D2
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0



This page does not contain notes.

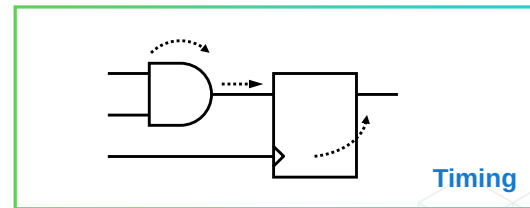
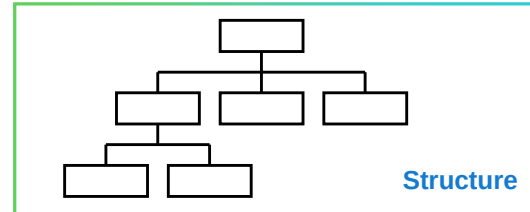
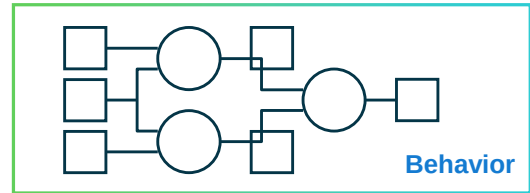
What Is Hardware Description Language (HDL)?

HDL is a programming language with special constructs for modeling hardware, concurrency, and timing.

HDL supports designs at multiple levels of abstraction:

- **Behavioral modeling**
 - Sequential behavior
 - Parallel behavior
- **Structural modeling**
 - Hardware component hierarchy
 - Software subroutine hierarchy
- **Time modeling**
 - An HDL has to model propagation delays, clock periods, and timing checks

HDL typically supports the simulation of estimated design timing.



5

© Cadence Design Systems, Inc. All rights reserved.

cadence

An HDL is similar to a procedural programming language but also contains constructs to describe digital electronic systems. An HDL contains features and constructs to support descriptions of the following:

- **Behavior** – Both serial and parallel. In serial behavior, you pass the output of one functional block to the input of another, which is similar to the behavior of a conventional software language. However, in parallel behavior, you can pass a block output to the inputs of a number of blocks acting in parallel where many separate events happen at the same moment in time.
- **Structure** – Both physical, such as hierarchical block diagrams and component netlists, and software, such as subroutines. This allows you to describe large, complex systems and manage their complexity.
- **Time** – Programming languages have no concept of time. An HDL has to model propagation delays, clock periods, and timing checks.

An HDL typically supports multiple abstraction levels. You can describe hardware behaviorally, without and with sufficient detail for logic synthesis, and as a structured netlist of predefined components that can themselves be as simple as a transistor and as complex as another behavioral design.

Why Use a Hardware Description Language?

The benefits of using an HDL include the following:

- You write HDL in plain old ASCII text:
 - Capture the design quickly and easily manage modifications.
- You can design at a higher level of abstraction:
 - Easily explore design alternatives.
 - Find problems earlier in the design cycle.
- Use of a standard HDL enables design reuse:
 - Between design teams and partners and through the design flow.
 - No re-entry, reformatting, or translation.
- The description is independent of the implementation:
 - You can delay the choice of implementation technology.
 - You can more easily make architectural and functional changes.
 - You can more easily adapt the design to future projects.



The benefits of using an HDL include the following:

- You can capture and modify the design more quickly in an HDL than by schematic capture.
- Earlier design capture at the higher abstraction level means earlier simulation, which facilitates design alternative exploration to produce a more optimal architecture and partitioning and allows a more thorough verification.
- Use of a standard HDL facilitates the reuse of designs from previous projects or from commercial Intellectual Property (IP) providers. You can move or switch between different tools and vendors without re-entry, reformatting, or translation of the design description.
- Your RT-level implementation is almost 100% independent of the implementation technology, so you can defer selection of the target ASIC or FPGA technology until the design is mainly entered, and you can switch technology or vendor with a minimum of a redesign.

Before and After HDLs

Before the Verilog, VHDL or SystemVerilog languages were invented, designs were:

- Small and much less complex.
- Captured using schematics.
- Simulated with SPICE*.

With the advent of these HDL languages, designs are:

- Much larger and more complex.
- Designed using software.
- Simulated with a Verilog/SystemVerilog or VHDL simulator.



**Simulation Program with Integrated Circuit Emphasis*

An HDL Supports Multiple Levels of Abstraction

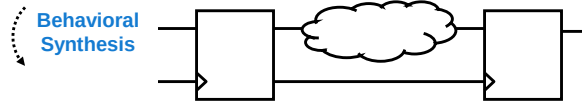
Behavioral level

- Algorithms



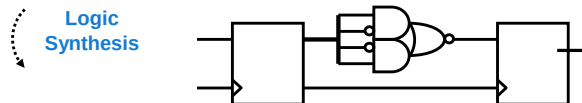
Register Transfer Level (RTL)

- Nets and registers



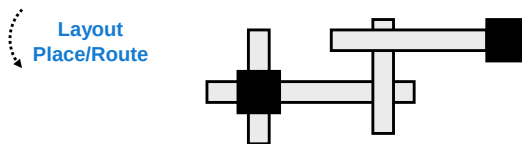
Gate level

- Built-in and user-defined primitives



Switch level

- Built-in switch primitives



8 © Cadence Design Systems, Inc. All rights reserved.

cadence

You can describe a system as a group of hierarchical models of varying amounts of detail.

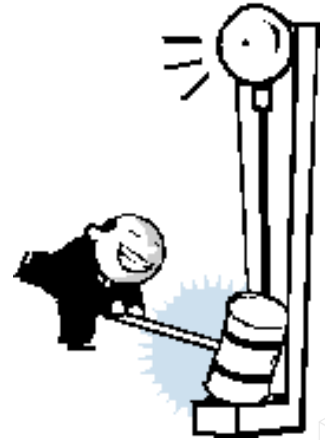
An HDL supports multiple levels of such detail. The three main levels of abstraction are listed and described below:

- The behavioral level:
 - You describe the system using mathematical equations.
 - You can omit timing – the system may simulate in zero time like a software program.
- The Register Transfer Level (RTL):
 - You partition the system into combinational and sequential logic, using constructs and coding styles supported by logic synthesis.
 - You define timing in terms of cycles based on one or more defined clock(s).
- The structural level:
 - You instantiate and interconnect predefined components.
 - Can include vendor-provided macrocells.
 - Can include logic primitives built into the language.

Abstraction Level Tradeoffs

Simulation effort is proportional to detail:

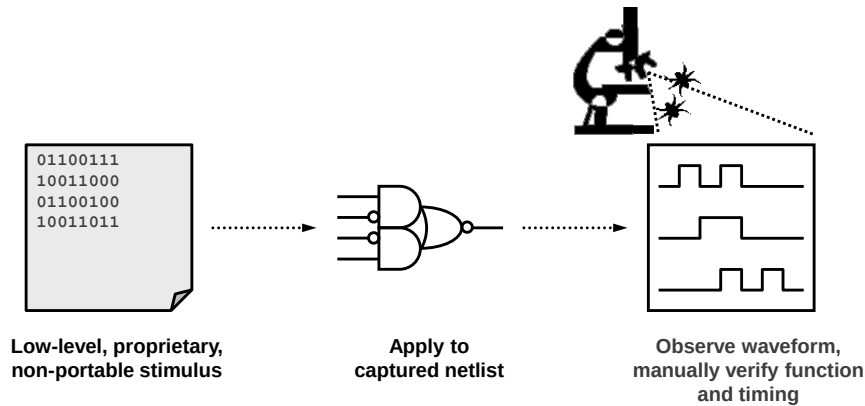
- Behavioral level
 - Algorithms
 - Register Transfer Level (RTL)
 - Nets and registers
 - Gate level
 - Built-in and user-defined primitives
 - Switch level
 - Built-in switch primitives
- Less detail
 - Faster design entry
 - Faster simulation
 - More detail
 - Slower design entry
 - Slower simulation



Users of an HDL model at different levels of abstraction:

- Block architects model functionality at the behavioral level for maximum simulation speed and for the ease with which they can quickly modify the architecture as they explore alternatives.
- Block implementers refine the functional blocks to the RT level for a logic synthesis tool.
- Library developers model cells at the gate level for simulation and the physical level for place and route tools.

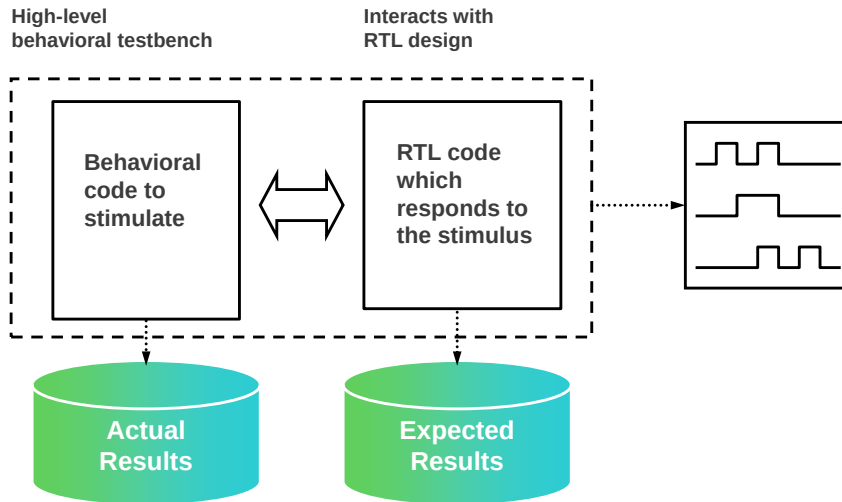
Legacy Schematic-Based Design



Let us compare to an HDL ...

Designers used to capture the design intent using a proprietary schematic capture tool, manually developed the test stimulus in a proprietary tabular format, and simulated the design using a proprietary simulator. Those who had money to burn had graphical displays to examine the results, but most of us had plain old text terminals so that we could examine the results only with 0 and 1 characters. We did not use the Z and X characters because those values do not exist in hardware, and we used simulation only to generate expected results for a device test machine.

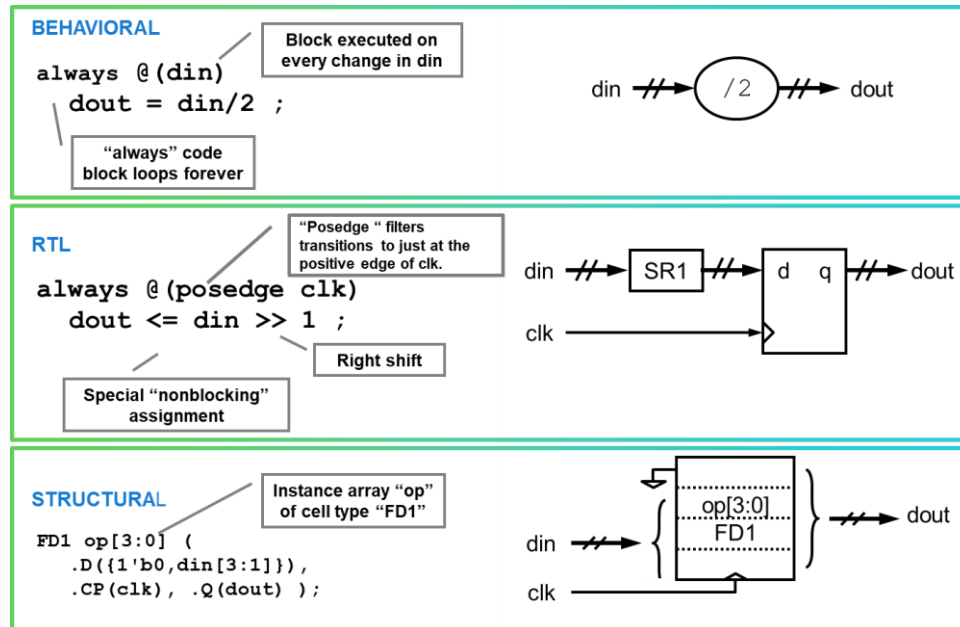
HDL-Based Simulation



- Testbench can be self-checking
- Design and tests vendor independent

In more recent times, we capture the design intent and its test using the same standard HDL and simulate them together using a choice of tools running on a wide choice of third-party platforms. We have high-definition, wide-screen displays that we do not have to stare at quite so much because our tests, to a large extent, pinpoint any problems with a high degree of accuracy.

Abstraction Level Example: Divide by 2 Operation



12 © Cadence Design Systems, Inc. All rights reserved.

cadence

These code fragments illustrate the three main levels of abstraction:

- The behavioral level of abstraction describes the design behavior with no hint about how the operation is implemented.
- The RT level of abstraction describes the design behavior with sufficient detail that logic synthesis can infer an implementation involving an edge-triggered storage device.
- The structural level of abstraction instantiates and connects a predefined storage device from a macro library, not caring how the component is itself implemented.

What Is RTL Coding?

Converting the design specification set of rules into a high-level design using a Hardware Description Language such as Verilog, VHDL, SystemVerilog, SystemC, etc., is called RTL Coding.

- RTL is an acronym for “Register Transfer Level.”
- Instead of using schematic capture like in the old days to design a complex chip, designers now use an HDL.

```
module mux (output logic y,  
            input logic s,  
            input logic i1,  
            input logic i0);  
  
    always_comb  
        if (s==1)  
            y = i1;  
        else  
            y = i0;  
  
endmodule
```

This page does not contain notes.



Module 5-2

Hardware Implementation: Data Representation

cādence[®]

This page does not contain notes.

Objectives

In this module, you will:

- Look at the way in which data is represented in binary.

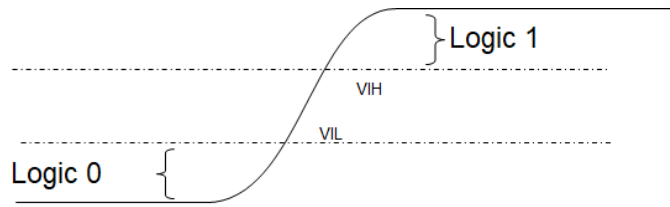
Topics include:

- Unsigned numbers
- Signed numbers
- Other signed data formats
- Floating point representations
- Binary coded decimal
- Gray code



In this submodule, we look at the hardware representation of data in binary, including signed and unsigned numbers, floating point representations, and some selected, application-specific representations in Binary Coded Decimal and Gray Code

Binary Numbers



Why use binary numbers?

- Circuits only discriminate between two states: 1 and 0.
- Transistors need only to operate as switches:
 - on – short circuit
 - off – open circuit

Many options for representing data.

- Corresponding mechanisms for manipulating data.

V_{IH} is a higher voltage threshold. Any voltage greater than this is considered a logic 1.

V_{IL} is a lower voltage threshold. Any voltage lower than this is considered a logic 1.

Binary Bus

A binary number is made up of individual bits

In hardware, a collection of related bits is a bus

- Also called a vector or array

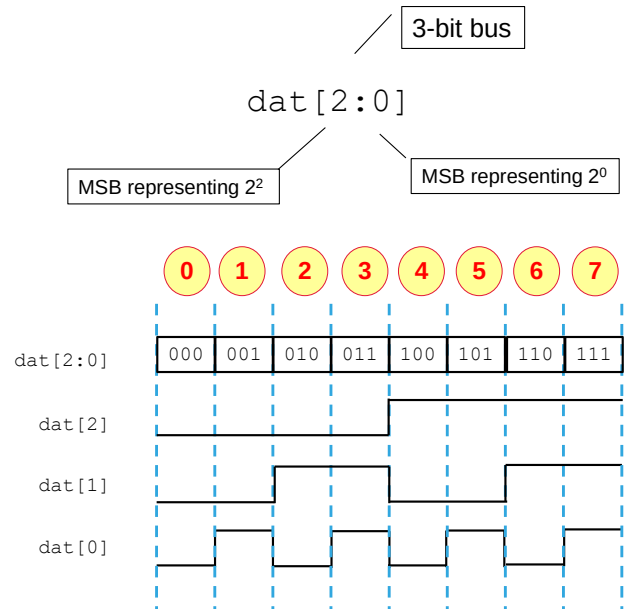
Busess are usually of a standard width

- e.g. 4, 8, 16, 32, 64

Position of a bit in the bus determines its value

- Bus is indexed by an integer
- The left-most bit is the Most-Significant Bit (MSB)
- Right-most is the Least-Significant Bit (LSB)

	Binary		
	2^2	2^1	2^0
Decimal	4	2	1



Individual binary bits are collated into a bus. Each binary bit has a position in the bus, indexed by an integer. A bit's index defines the value of the individual bit's contribution to the overall bus value.

Busess are usually of standard widths which are usually a power of 2, e.g. 32 bits or 64 bits.

The lowest index of a bus is 0 by convention, and bit 0 is usually the Least-Significant Bit (LSB) representing $2^0 = 1$. The highest index of a bit is usually the Most-Significant Bit (MSB). Bit ordering is usually from MSB to LSB (e.g. `dat[2:0]`) although exceptions can occur, and you may see the reverse notation (`dat[0:2]`). In HDL's, it may be common to see both notations used in the same design, so you need to check carefully.

The example shows a 3-bit bus, indexed in descending order `[3:0]` with the right-most bit as the LSB. Therefore `dat[0]` represents 2^0 , `dat[1]` 2^1 and `dat[2]` 2^2 , and as such, 101 in binary represents $4+1 = 5$ in decimal.

Binary Representation

Many more digits are required for binary.

Hexadecimal is more convenient to describe binary numbers:

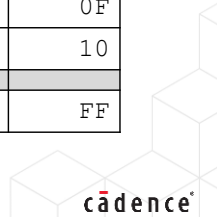
- Each hex digit is 4 binary bits
- Decimal digits for 0-9
- Letters A-F for 10-15

Specific binary widths are named:

Bits	Name
4	nibble, half-byte
8	byte
16	word*

*or data width of a processor, e.g., 32 or 64-bits

Decimal	Binary								Hex
	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
	128	64	32	16	8	4	2	1	
0	0	0	0	0	0	0	0	0	00
1	0	0	0	0	0	0	0	1	01
2	0	0	0	0	0	0	1	0	02
9	0	0	0	0	1	0	0	1	09
10	0	0	0	0	1	0	1	0	0A
11	0	0	0	0	1	0	1	1	0B
15	0	0	0	0	1	1	1	1	0F
16	0	0	0	1	0	0	0	0	10
255	1	1	1	1	1	1	1	1	FF



Certain binary widths may have specific names, although the names may not be consistently used.

An 8-bit binary is referred to as a byte with 4 bits as a nibble or half-byte. Nibbles can be represented in hex. Most PCs and game consoles are based on an Intel X86 processor architecture, which designates 16 bits as a WORD. Multiples of 16 bits are references as multiples of words, e.g., 32 bits is DWORD (double word) and 64-bit QWORD. Microsoft Windows still uses these terms.

However, in hardware, strictly speaking, a WORD is the fixed-width data handled by the processor. Therefore, for a modern 64-bit processor, a word is 64 bits.

Binary requires many more digits to represent numbers, as every digit is a power of 2 rather than, as in decimal, a power of 10. For example, we need 8 binary digits to represent decimal numbers from 0 to 255. It is more convenient to describe binary numbers using hexadecimal (or hex), where each hex digit represents 4 binary bits. The range of 4 bits is 0 to 15, and hex uses decimal digits for 0-9 and the letters A to F for 10 to 15. The table shows selected 8-bit values in decimal, binary, and hex. Specific binary widths are named. 8 bits is a byte, and 4 bits, a single hex digit, is a nibble or half-byte. 16 bits can be referred to as a word, and multiples of 16 bits as multiples of the word; for example, 32 bits is a double word or DWORD. However, this naming is inconsistent as a word can also refer to the data width of a processor, which could be 32 or 64 bits.

Binary Unsigned Data

N bit *unsigned* binary number has a range 0 to 2^N-1 .

In an implementation, the “width” of a word must be sufficient to cope with the biggest number encountered.

Number of bits(N)	Max value (2^N-1)
8	255
16	65535
32	4,294,967,295

Question: What happens if a word is not wide enough?



For unsigned, positive binary numbers, the number of bits defines the range of numbers represented. So, 8 bits have the range 0 to 255 decimal and 16 bits 0 to 65535. What happens if a bus or connection is not wide enough to store the required data? Well, typically, the data is truncated, and information is lost. For example, if you add one to a byte containing hex FF (decimal 255), then we need 9 bits to store the result, but with only 8 bits, the result rolls over to 0.

Binary Signed Data

2s complement representation for negative numbers.

- MSB determines sign of number.
 - "0" for positive, "1" for negative.
- To find $-number$, subtract from 0 and truncate to N bits.
 - Easiest way to do this is invert number and add 1.

+6 $\rightarrow$ 0110

invert 1001

add 1 1010

-6 $\rightarrow$ 1010

$$-number = number + 1$$

Number of bits(N)	Range $-(2^{N-1})$ to $+(2^{N-1}-1)$
8	-128 to 127
16	-32,768 to +32,767
32	-2,147,483,648 to +2,147,483,647



Note: Any pattern of bits is a 2s complement number, which determines its arithmetic value.

To represent both positive and negative numbers called signed data, the most common format is 2's complement. Here the Most Significant binary Bit (or MSB) is reserved purely to define the sign of the number. If 0, the number is positive, and if 1, it is negative. To convert a positive binary value to negative, we subtract the value from 0 and truncate the result to the number of bus bits. An equivalent and easier way is to invert the binary value and add 1. So, to convert +6 to negative, we invert 0110 to 1001 and add 1 to give 1010. As the MSB is reserved purely for the sign information, this restricts the range of a signed bus. So 8bits represents from -128 to +127. Obviously, the hardware needs to understand if a binary value is a 2s complement or unsigned data and interpret it correctly.

Examples: Signed and Unsigned

For 8-bit numbers:

Number	Binary	2s Complement	Number
128	10000000	10000000	-128 (!)
0	00000000	00000000	0
1	00000001	11111111	-1
127	01111111	10000001	-127

note: no +ve equivalent

0 assumed to be positive

biggest positive number → 127

Unsigned Arithmetic

```

1111000 120
  1000 + 8
10000000 128
  
```

Signed Arithmetic

```

01111000 120
11111000 - 8
01110000 112
  
```

Extra bit required for sign information

Signed Arithmetic

```

01111000 120
  1000 + 8
10000000 -128!
  
```

Overflow!!

21 © Cadence Design Systems, Inc. All rights reserved.

cadence

These examples represent limiting cases of classes Max, Min, Zero, and a few other cases. Note that $-2N-1$ converts to itself (no positive equivalent) as does 0.

Here are some signed and unsigned examples using 8-bit data. The MSB sign bit in 2s complement restricts the range, so an 8-bit signed bus can hold -128 but not +128. The biggest positive number is 127. Signed and unsigned representations of 0 are identical, and as the signed MSB is not 1, zero is assumed to be positive in the 2s complement. We need to be careful about bit widths in arithmetic. In the unsigned example, adding 120 to 8 gives 128, which requires 9 bits. If the arithmetic was implemented in only 8 bits, we would lose the MSB, and the result would be 0. In signed arithmetic, we can add a 2s complement number to perform a subtraction. So, adding 2s complement of 8 to 120 subtracts 8. Here we rely on truncation to 8 bits to maintain the sign of the result. However, overflow can still be a problem in signed arithmetic. The last example shows that adding +8 to 120 overflows the result into the MSB, overwriting the sign information, and giving the result -128.

Other Signed Data Representations

1s complement

- Negative number is a simple inversion of a positive number:
 - Simple to implement
 - Arithmetic has exceptions

Off-set binary

- Most negative number in the range is treated as 0:
 - Arithmetic simplified
 - Conversion difficult for large ranges

Decimal	-128	-37	0	37	127
Offset Binary	00000000	01011011	10000000	10100101	11111111

22

© Cadence Design Systems, Inc. All rights reserved.



1s complement arithmetic is not as straight forward as 2s complement.

Try $1 - (-1)$ in 1s complement.

There are other signed number representations, but these are rarely used. 1s complement simply inverts a positive number to make it negative. This is simple to implement but gives issues in arithmetic. For example, there are positive zero and negative zero as two different values. Offset binary treats the most negative number in a range as zero and offsets all the other values from this. For the range -128 to 127, -128 is represented as zero, and all values increment from this. The arithmetic is simple, positive values only, and offset is good for ranges where bounds are not powers of two, but conversion into and out of the offset coding can be difficult for wide ranges.

Floating Point: IEEE Standard 754



IEEE single format consists of 3 fields in 32 bit word

- F - 23-bit fraction
- E - 8-bit biased exponent
- S - single bit sign

Represents data via the equation:

$$(-1)^S \times 2^{E-127} \times 1.F$$



Single-Format Bit Pattern Value

$0 < e < 255$	$(-1)^s \times 2^{e-127} \times 1.f$ (normal numbers)
$e = 0; f \neq 0$	$(-1)^s \times 2^{-126} \times 0.f$ (subnormal numbers)
(at least one bit in f is nonzero)	
$e = 0; f = 0$	$(-1)^s \times 0.0$ (signed zero)
$s = 0; e = 255; f = 0$	+INF (positive infinity)
$s = 1; e = 255; f = 0$	-INF (negative infinity)
$s = u; e = 255; f \neq 0$	NaN (Not-a-Number)
(at least one bit in f is nonzero)	

For large and fractional number ranges, we need more efficient formats. One example is the floating-point IEEE 754 standard. For 32 bits, the format defines the lowest 23 bits, F as the fractional bits. Then the next 8 bits are a biased exponent E, in that, you subtract 127 from E and raise 2 to the result. The MSB S is the sign bit. The equation shows how to convert IEEE754 to decimal. Remember, the fractional bits are still powers of 2, so 0.1 binary is $0 + 2^{-1} = 0.5$ decimal.

Examples: Single Format

Number	Bit Pattern (hex)	Decimal Value
+0	00000000	0.0
-0	80000000	-0.0
1	3f800000	1.0
max value	7f7fffff	3.40282347e+38
min normal	00800000	1.17549435e-38
min subnormal	00000001	1.40129846e-45

IEEE 754 also defines:

- Double precision format (64-bit)
 - 52-bit fraction; 11-bit biased exponent; 1-bit sign
- Accuracy requirements on operations and conversions
- Floating-point exceptions

24

© Cadence Design Systems, Inc. All rights reserved.



Accuracy requirements on floating-point operations include:

- Add, subtract, multiply, divide, square root, remainder, and round numbers in floating-point format to integer values, convert between different floating-point formats, convert between floating-point and integer formats, and compare.
- Five types of floating-point exceptions: invalid operation, division by zero, overflow, underflow, and inexact.

Here are some IEEE 754 examples in 32 bits (called binary32). IEEE 754 can represent normal numbers in the range of approximately 3.4×10^{38} to 1.17×10^{-38} , although there is a subnormal mode that allows numbers down to 1.4×10^{-45} . Binary32 is a single precision format, but IEEE754 also defines a 64-bit double precision format with 52 fractional bits. The standard not only defines the number format but also accuracy requirements on arithmetic operations, such as multiplication, division and square roots, and conversions to and from integers and between different floating-point formats. The standard also defines the result, and indication, of arithmetic exceptions such as dividing by zero, overflow and underflow.

Binary Coded Decimal

- Numbers can be represented as a “coded” equivalent of decimal.
- Each decimal digit (0...9) is stored as 4 bits.
- A N-digit number is made up from 4*N bits.

Binary	Decimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010...	<i>Not</i>
1111	<i>used</i>

Example

9832 = 1001 1000 0011 0010
 9 8 3 2



Multi-digit operations/arithmetic are possible but can be cumbersome.

Some data formats are more application specific. For example, Binary Coded Decimal, or BCD. This format uses 4bits to represent a single decimal digit in binary format, with the values 1010 to 1111 not used. Multiple 4-bit nibbles are used for multi-digit decimal numbers. BCD's advantages are in its close correspondence to decimal. So, the format is ideal for clock and counter displays. A disadvantage is that arithmetic on multi-digit values is more complex.

Gray Code

Gray code is a cyclic code.

- Only one-bit changes from state to state.
 - No erroneous states during transitions.
 - Lower power used in transitions.
- Simple relationship for conversion:
 - $\text{gray}(N) = \text{binary}(N+1) \text{ xor } \text{binary}(N)$

Binary		Gray
000		000
001		001
010		011
011		010
100		110
101		111
110		101
111		100



26 © Cadence Design Systems, Inc. All rights reserved.



In Gray code, only one bit changes state from state to state:

- No indeterminate or false states
- Few bits changing, therefore lower power utilisation

There is a simple relationship to convert binary to gray, e.g.:

- $\text{gray}(0) = \text{binary}(1) \text{ xor } \text{binary}(0);$
- $\text{gray}(1) = \text{binary}(2) \text{ xor } \text{binary}(1);$

Gray code is used mainly for simple counters and state vector encoding.

Another application-specific format example is Gray code. One disadvantage of counting in binary is that multiple bits must change in the transition from one value to the next. For example, counting from 3 to 4, three bits must toggle. Hardware delays may mean that not all bits change at the same time, leading to intermittent or false values during the transition. With Gray coding, only 1-bit changes from state to state, giving a safer transition. Fewer changing bits also give a lower power utilization. There is a simple conversion algorithm from binary to Gray code, as shown.

Summary

In this module, you learned about:

- A wide choice of formats for representing data in binary.
- The choice for particular implementation is determined by:
 - Range of data
 - Operations to be carried out on data
 - Need for standard formats
 - Destination of data
 - Power and reliability requirements



In summary, there is a wide choice of formats for binary data representation. The choice of format is determined by the range of values to be represented; the data operations required, whether you need standard formats; what the data is used for and the power and reliability requirements.